

APSSDC AWS & DEVOPS INTERNSHIP PROJECT

PlaceSync

Student Placement Tracker

*A decoupled Node.js application, automatically deployed on AWS
using Terraform for end-to-end Infrastructure as Code.*

PRESENTED BY

Pattan Mahaboob Subhani

Papadasu Satish

Gangavarapu Tharun Kumar

Ikkurthi Lakshmi Narayana

Netuluri Sphurthi

Agenda

- 1 Problem Statement
- 2 Solution Overview
- 3 Key Features
- 4 Technology Stack
- 5 AWS Services Used
- 6 Architecture
- 7 Infrastructure as Code & Deployment
- 8 Local Development Setup
- 9 Future Scope
- 10 Conclusion

Problem Statement



Placement information is scattered across notices, emails, and spreadsheets — no single source of truth for students.



Students have no easy way to view placement drives, company details, or track their own application status.



Coordinators manage company and drive data manually, which does not scale as the number of drives grows.



Interns need hands-on experience provisioning real AWS infrastructure, not just building an application.

Solution Overview

PlaceSync is architected as two decoupled services — a static frontend and a Node.js/Express REST API — provisioned on AWS entirely through Infrastructure as Code with Terraform.



Decoupled Architecture

Frontend (static HTML/CSS/JS) and backend (Express REST API) are independently deployable services.



Dual Persistence

The backend supports a local JSON file for development, and AWS DynamoDB for production.



Infrastructure as Code

All AWS resources — S3, EC2, CloudFront, DynamoDB, IAM — are provisioned via Terraform.

Key Features



Student Portal

`index.html`

- View placement drives & company details
- Apply to a company
- Track application status



Admin Dashboard

`admin.html`

- Manage placement drives
- Add / update company records
- Oversee student applications



Backend API

`backend/src`

- REST endpoints for drives & applications
- Local JSON or DynamoDB persistence
- CORS-free via CloudFront routing

Technology Stack



Node.js + Express

Backend REST API



HTML / CSS / JS

Decoupled static frontend



JSON / DynamoDB

Local dev vs. production persistence



Terraform (HCL)

Infrastructure as Code



AWS CLI

Deployment tooling



concurrently

Local orchestration of both services



Git & GitHub

Version control & source of truth



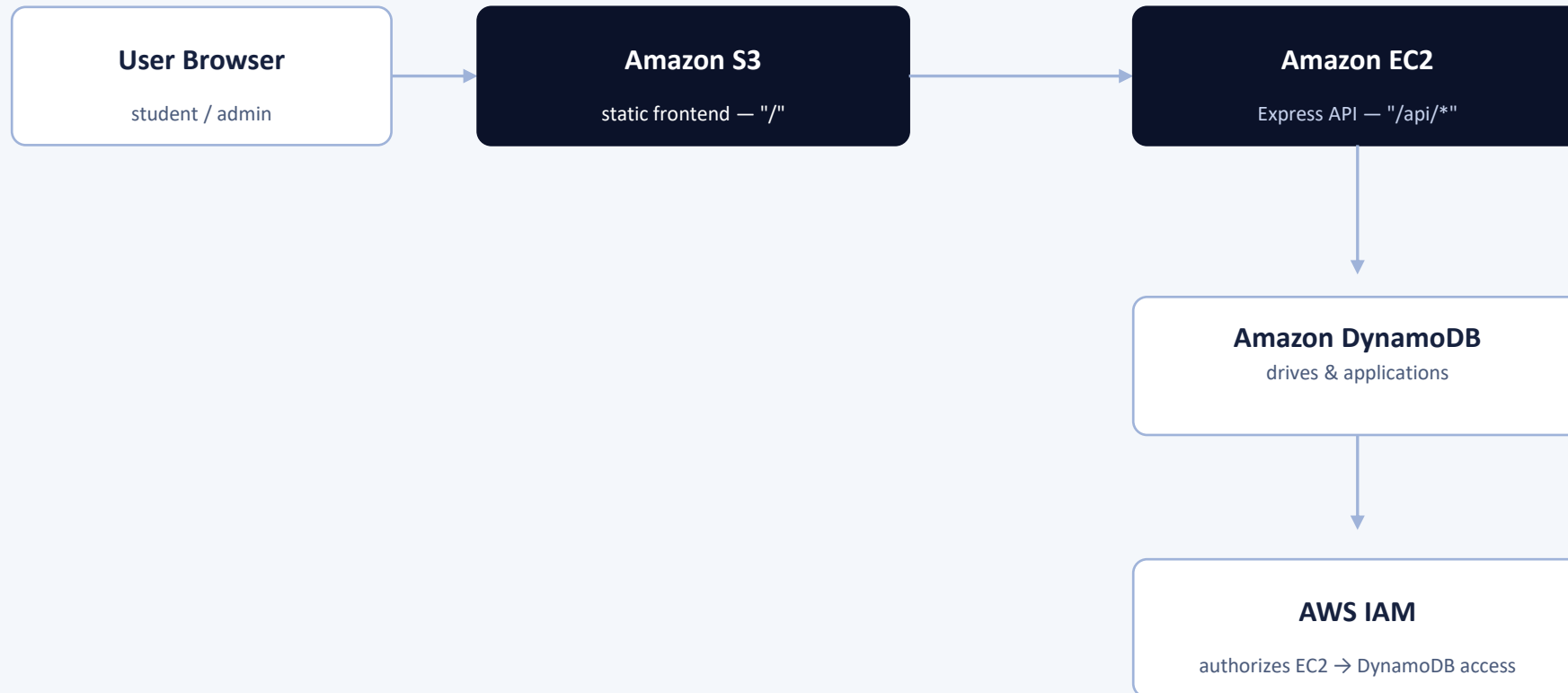
npm workspaces

Root-level dependency management

AWS Services Used

#	AWS Service	Purpose in PlaceSync
1	Amazon S3	Hosts the static frontend web client
2	Amazon EC2	Hosts the Node.js/Express backend API service
3	Amazon DynamoDB	Key-value persistent storage for placement drives and applications
4	Amazon CloudFront	Routes "/" to S3 and "/api/*" to EC2 under one domain — no CORS needed, plus CDN distribution
5	AWS IAM	Authorizes the EC2 instance to securely access DynamoDB tables

Architecture



Infrastructure as Code & Deployment



Local Development Setup

Prerequisite: Node.js version 18.0.0 or later

Install dependencies

```
npm install && npm run install:all
```

Run both services

```
npm run dev
```

- Frontend served at <http://localhost:5000>
- Backend API runs at <http://localhost:3000>
- Local backend defaults to a JSON file (`backend/src/data/db.json`)
- CORS is pre-configured to allow requests from port 5000

Future Scope

-  Automate provisioning and deployment with GitHub Actions CI/CD
-  Add Amazon CloudWatch dashboards & alarms for backend monitoring
-  Integrate Amazon SNS for deployment and alarm notifications
-  Add authentication for student and admin roles
-  Attach a custom domain with HTTPS via an ACM certificate on CloudFront

Conclusion



PlaceSync demonstrates a decoupled, cloud-native architecture for a real placement-tracking use case.



All infrastructure is provisioned as code with Terraform — S3, EC2, CloudFront, DynamoDB, and IAM.



Dual persistence (local JSON / DynamoDB) allows development without any AWS dependency.



The project establishes a clear foundation for CI/CD automation, monitoring, and alerting.

Thank You

Questions & Discussion — PlaceSync